

To put this prediction in perspective, each year the Gordon Bell Prize acknowledges advances in parallelism, including the fastest real program (highest MFLOPS). In 1989, the winner used an eight-processor Cray Y-MP to run at 1680 MFLOPS. On the basis of these numbers, multiprocessors and programs would have to have improved by a factor of 3.6 each year for the fastest program to achieve 1 TFLOPS in 1995. In 1999, the first Gordon Bell prize winner crossed the 1 TFLOPS bar. Using a 5832-processor IBM RS/6000 SST system designed specially for Livermore Laboratories, they achieved 1.18 TFLOPS on a shock wave simulation. This ratio represents a year-to-year improvement of 1.93, which is still quite impressive.

What has been recognized since the 1990s is that although we may have the technology to build a TFLOPS multiprocessor, it is not clear that the machine is cost-effective, except perhaps for a few very specialized and critically important applications related to national security. We estimated in 1990 that achieving 1 TFLOPS would require a machine with about 5000 processors and would cost about \$100 million. The 5832-processor IBM system at Livermore cost \$110 million. As might be expected, improvements in the performance of individual microprocessors both in cost and performance directly affect the cost and performance of large-scale multiprocessors, but a 5000-processor system will cost more than 5000 times the price of a desktop system using the same processor. Since that time, much faster multiprocessors have been built, but the major improvements have increasingly come from the processors in the past 5 years, rather than fundamental breakthroughs in parallel architecture.

The second Bell prediction concerned the number of data streams in supercomputers shipped in 1995. Danny Hillis believed that although supercomputers with a small number of data streams might be the best sellers, the biggest multiprocessors would be multiprocessors with many data streams, and these would perform the bulk of the computations. Bell bet Hillis that in the last quarter of calendar year 1995, more sustained MFLOPS would be shipped in multiprocessors using few data streams (<100) rather than many data streams (>1000). This bet concerned only supercomputers, defined as multiprocessors costing more than \$1 million and used for scientific applications. Sustained MFLOPS was defined for this bet as the number of floating-point operations per month, so availability of multiprocessors affects their rating.

In 1989, when this bet was made, it was totally unclear who would win. In 1995, a survey of the current publicly known supercomputers showed only six multiprocessors in existence in the world with more than 1000 data streams, so Bell's prediction was a clear winner. In fact, in 1995, much smaller microprocessor-based multiprocessors (<20 processors) were becoming dominant.

In 1995, a survey of the 500 highest-performance multiprocessors in use (based on Linpack ratings), called the Top 500, showed that the largest number of multiprocessors were bus-based shared memory multiprocessors! By 2005,

various clusters or multicomputers played a large role. For example, in the top 25 systems, 11 were custom clusters, such as the IBM Blue Gene system or the Cray XT3, 10 were clusters of shared memory multiprocessors (both using distributed and centralized memory), and the remaining four were clusters built using PCs with an off-the-shelf interconnect.

More Recent Advances and Developments

With the primary exception of the parallel vector multiprocessors and more recently of the IBM Blue Gene design, all other modern MIMD computers have been built from off-the-shelf microprocessors using a bus and logically central memory or an interconnection network and a distributed memory. A number of experimental multiprocessors built in the 1980s further refined and enhanced the concepts that form the basis for many of today's multiprocessors.

The Development of Bus-Based Coherent Multiprocessors

Although very large mainframes were built with multiple processors in the 1960s and 1970s, multiprocessors did not become highly successful until the 1980s. [Bell \[1985\]](#) suggests the key was that the smaller size of the microprocessor allowed the memory bus to replace the interconnection network hardware and that portable operating systems meant that multiprocessor projects no longer required the invention of a new operating system. In this paper, Bell defined the terms multiprocessor and multicomputer and set the stage for two different approaches to building larger-scale multiprocessors. The first bus-based multiprocessor with snooping caches was the Synapse N + 1 in 1984.

The early 1990s saw the beginning of an expansion of such systems with the use of very wide, high-speed buses (the SGI Challenge system used a 256-bit, packet-oriented bus supporting up to eight processor boards and 32 processors) and later the use of multiple buses and crossbar interconnects, for example, in the Sun SPARCCenter and Enterprise systems. In 2001, the Sun Enterprise servers represented the primary example of large-scale (>16 processors), symmetric multiprocessors in active use.

Toward Large-Scale Multiprocessors

In the effort to build large-scale multiprocessors, two different directions were explored: message-passing multicomputers and scalable shared memory multiprocessors. Although there had been many attempts to build mesh and hypercube-connected multiprocessors, one of the first multiprocessors to successfully bring together all the pieces was the Cosmic Cube built at Caltech [\[Seitz, 1985\]](#). It introduced important advances in routing and interconnect technology and substantially reduced the cost of the interconnect, which helped make the multicomputer viable. The Intel iPSC 860, a hypercube-connected collection of i860s,

was based on these ideas. More recent multiprocessors, such as the Intel Paragon, have used networks with lower dimensionality and higher individual links. The Paragon also employed a separate i860 as a communications controller in each node, although a number of users have found it better to use both i860 processors for computation as well as communication. The Thinking Multiprocessors CM-5 made use of off-the-shelf microprocessors. It provided user-level access to the communication channel, significantly improving communication latency. In 1995, these two multiprocessors represented the state of the art in message-passing multicomputers.

Clusters

Clusters were probably “invented” in the 1960s by customers who could not fit all their work on one computer, or who needed a backup machine in case of failure of the primary machine [Pfister, 1998]. Tandem introduced a 16-node cluster in 1975. Digital followed with VAX clusters, introduced in 1984. They were originally independent computers that shared I/O devices, requiring a distributed operating system to coordinate activity. Soon they had communication links between computers, in part so that the computers could be geographically distributed to increase availability in case of a disaster at a single site. Users log on to the cluster and are unaware of which machine they are using. DEC (now HP) sold more than 25,000 clusters by 1993. Other early companies were Tandem (now HP) and IBM (still IBM). Today, virtually every company has cluster products. Most of these products are aimed at availability, with performance scaling as a secondary benefit.

Scientific computing on clusters emerged as a competitor to MPPs. In 1993, the Beowulf project started with the goal of fulfilling NASA's desire for a 1-GFLOPS computer for less than \$50,000. In 1994, a 16-node cluster built from off-the-shelf PCs using 80486s achieved that goal. This emphasis led to a variety of software interfaces to make it easier to submit, coordinate, and debug large programs or a large number of independent programs.

Efforts were made to reduce latency of communication in clusters as well as to increase bandwidth, and several research projects worked on that problem. (One commercial result of the low-latency research was the VI interface standard, which has been embraced by Infiniband, discussed below.) Low latency then proved useful in other applications. For example, in 1997 a cluster of 100 UltraSPARC desktop computers at U.C. Berkeley, connected by 160 MB/sec per link Myrinet switches, was used to set world records in database sort (sorting 8.6 GB of data originally on disk in 1 minute) and in cracking an encrypted message (taking just 3.5 hours to decipher a 40-bit DES key).

This research project, called Network of Workstations, also developed the Inktomi search engine, which led to a start-up company with the same name.

Google followed the example of Inktomi to build search engines from clusters of desktop computers rather than large-scale SMPs, which was the strategy of the leading search engine, Alta Vista, that Google took over. In 2013, virtually all Internet services rely on clusters to serve their millions of customers.

Clusters are also very popular with scientists. One reason is their low cost, which enables individual scientists or small groups to own a cluster dedicated to their programs. Such clusters can get results faster than waiting in the long job queues of the shared MPPs at supercomputer centers, which can stretch to weeks.

For those interested in learning more, Pfister [1998] has written an entertaining book on clusters.

Recent Trends in Large-Scale Multiprocessors

In the mid-to-late 1990s, it became clear that the hoped-for growth in the market for ultralarge-scale parallel computing was unlikely to occur. Without this market growth, it became increasingly obvious that the high-end parallel computing market was too small to support the costs of highly customized hardware and software designed for a small market. Perhaps the most important trend to come out of this observation was that clustering would be used to reach the highest levels of performance. There are now three general classes of large-scale multiprocessors:

1. Clusters that integrate standard desktop motherboards using interconnection technology, such as Myrinet or Infiniban
2. Multicomputers built from standard microprocessors configured into processing elements and connected with a custom interconnect, such as the IBM Blue Gene
3. Clusters of small-scale shared memory computers, possibly with vector support, including the Earth Simulator

The IBM Blue Gene is the most interesting of these designs, since its rationale parallels the underlying causes of the recent trend towards multicore in uniprocessor architectures. Blue Gene started as a research project within IBM aimed at the protein sequencing and folding problem. The Blue Gene designers observed that power was becoming an increasing concern in large-scale multiprocessors and that the performance/watt of processors from the embedded space was much better than those in the high-end uniprocessor space. If parallelism was the route to high performance, why not start with the most efficient building block and simply have more of them?

Thus, Blue Gene is constructed using a custom chip that includes an embedded PowerPC microprocessor offering half the performance of a high-end PowerPC, but at a much smaller fraction of the area and the power. This allows more system functions, including the global interconnect, to be integrated onto the same die.

The result is a highly replicable and efficient building block, allowing Blue Gene to reach much larger processor counts more efficiently. Instead of using stand-alone microprocessors or standard desktop boards as building blocks, Blue Gene uses processor cores. No doubt such an approach provides much greater efficiency. Whether the market can support the cost of a customized design and special software remains an open question.

In 2006, a Blue Gene processor at Lawrence Livermore with 32K processors held a factor of 2.6 lead in Linpack performance over the third-place system, which consisted of 20 SGI Altix 512-processor systems interconnected with Infiniband as a cluster.

Blue Gene's predecessor was an experimental machine, QCDOD, which pioneered the concept of a machine using a lower-power embedded microprocessor and tightly integrated interconnect to drive down the cost and power consumption of a node.

Looking Further

There is an almost unbounded amount of information on multiprocessors and multicomputers: conferences, journal papers, and even books seem to appear faster than any single person can absorb the ideas. No doubt many of these papers will go unnoticed—not unlike the past. Most of the major architecture conferences contain papers on multiprocessors. An annual conference, Supercomputing XY (where X and Y are the last two digits of the year), brings together users, architects, software developers, and vendors and publishes the proceedings in book, CD-ROM, and online (see www.scXY.org) form. Two major journals, *Journal of Parallel and Distributed Computing* and the *IEEE Transactions on Parallel and Distributed Systems*, contain papers on all aspects of parallel processing. Several books focusing on parallel processing are included in the following references, with Culler et al. [1998] being the most recent, large-scale effort. For years, Eugene Miya of NASA Ames has collected an online bibliography of parallel processing papers. The bibliography, which now contains more than 35,000 entries, is available online at: www.ira.uka.de/bibliography/Parallel/Eugene/index.html.

Asanovic et al. [2006] surveyed the wide-ranging challenges for the industry in this multicore challenge. That report may be helpful in understanding the depth of the various challenges.

In addition to documenting the discovery of concepts now used in practice, these references also provide descriptions of many ideas that have been explored and found wanting, as well as ideas whose time has just not yet come. Given the move toward multicore and multiprocessors as the future of high-performance computer architecture, we expect that many new approaches will be explored in the years ahead. A few of them will manage to solve the hardware and software problems that have been the key to using multiprocessing for the past 40 years!

Further Reading

Almasi, G. S. and A. Gottlieb [1989]. *Highly Parallel Computing*, Benjamin/Cummings, Redwood City, CA.

A textbook covering parallel computers.

Amdahl, G. M. [1967]. “Validity of the single processor approach to achieving large scale computing capabilities,” *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ (April), 483–85.

Written in response to the claims of the Illiac IV, this three-page article describes Amdahl’s law and gives the classic reply to arguments for abandoning the current form of computing.

Andrews, G. R. [1991]. *Concurrent Programming: Principles and Practice*, Benjamin/Cummings, Redwood City, CA.

A text that gives the principles of parallel programming.

Archibald, J. and J. -L. Baer [1986]. “Cache coherence protocols: Evaluation using a multiprocessor simulation model,” *ACM Trans. on Computer Systems* 4 4 (November), 273–98.

Classic survey paper of shared-bus cache coherence protocols.

Arpaci-Dusseau, A., R. Arpaci-Dusseau, D. Culler, J. Hellerstein, and D. Patterson [1997]. “High-performance sorting on networks of workstations,” *Proc. ACM SIG MOD/PODS Conference on Management of Data*, Tucson, AZ (May), 12–15.

How a world record sort was performed on a cluster, including architecture critique of the workstation and network interface. By April 1, 1997, they pushed the record to 8.6 GB in 1 minute and 2.2 seconds to sort 100 MB.

Asanovic, K., R. Bodik, B. C. Catanzaro, J. J. Gebis, P. Husbands, K. Keutzer, D. A. Patterson, W. L. Plishker, J. Shalf, S. W. Williams, and K. A. Yelick. [2006]. “The landscape of parallel computing research: A view from Berkeley.” Tech. Rep. UCB/EECS-2006-183, EECS Department, University of California, Berkeley (December 18).

Nicknamed the “Berkeley View,” this report lays out the landscape of the multicore challenge.

Bailey, D. H., E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrishnan, and S. K. Weeratunga. [1991]. “The NAS parallel benchmarks—summary and preliminary results,” *Proceedings of the 1991 ACM/IEEE conference on Super-computing* (August).

Describes the NAS parallel benchmarks.

Bell, C. G. [1985]. “Multis: A new class of multiprocessor computers,” *Science* 228(April 26), 462–467.

Distinguishes shared address and nonshared address multiprocessors based on micro processors.

Bienia, C., S. Kumar, J. P. Singh, and K. Li [2008]. “The PARSEC benchmark suite: characterization and architectural implications,” Princeton University Technical Report TR-81 1-008 (January).

Describes the PARSEC parallel benchmarks. Also see <http://parsec.cs.princeton.edu/>.

Bouknight, W. J., Denenberg, S. A., McIntyre, D. E., Randall, J. M., Sameh, A. H., and Slotnick, D. L. [1972]. The Illiac IV system, *Proceedings of the IEEE*, 60(4), 369–388.

This describes the most infamous SIMD supercomputer.

Culler, D. E. and J. P. Singh, with A. Gupta [1998]. *Parallel Computer Architecture*, Morgan Kaufmann, San Francisco.

A textbook on parallel computers.

Dongarra, J. J., J. R. Bunch, G. B. Moler, G. W. Stewart [1979]. *LINPACK Users' Guide*, Society for Industrial Mathematics.

The original document describing Linpack, which became a widely used parallel bench mark.

Falk, H. [1976]. "Reaching for the gigaflop", *IEEE Spectrum* 13: 10 (October), 65–70.

Chronicles the sad story of the Illiac IV: four times the cost and less than one-tenth the performance of original goals.

Flynn, M. J. [1966]. "Very high-speed computing systems", *Proc. IEEE* 54 12 (December), 1901–09.

Classic article showing SISD/SIMD/MISD/MIMD classifications.

Hennessy, J. and D. Patterson [2003]. Chapters 6 and 8 in *Computer Architecture: A Quantitative Approach*, third edition, Morgan Kaufmann Publishers, San Francisco.

A more in-depth coverage of a variety of multiprocessor and cluster topics, including programs and measurements.

Henning, J. L. [2007]. "SPEC CPU suite growth: an historical perspective", *Computer Architecture News* Vol. 35, no. 1 (March).

Gives the history of SPEC, including the use of SPECrate to measure performance on independent jobs, which is being used as a parallel benchmark.

Hillis, W. D. [1989]. *The connection machine*. The MIT Press.

PhD Dissertation that makes case for 1-bit SIMD computer.

Hord, R. M. [1982]. *The Illiac-IV, the First Supercomputer*, Computer Science Press, Rockville, MD.

A historical accounting of the Illiac IV project.

Hwang, K. [1993]. *Advanced Computer Architecture with Parallel Programming*, McGraw-Hill, New York.

Another textbook covering parallel computers.

Kozyrakis, C. and D. Patterson [2003]. "Scalable vector processors for embedded systems", *IEEE Micro* 23:6 (November–December), 36–45.

Examination of a vector architecture for the MIPS instruction set in media and signal processing.

Menabrea, L. F. [1842]. "Sketch of the analytical engine invented by Charles Babbage", *Bibliothèque Universelle de Genève* (October).

Certainly the earliest reference on multiprocessors, this mathematician made this comment while translating papers on Babbage's mechanical computer.

Pfister, G. F. [1998]. *In Search of Clusters: The Coming Battle in Lowly Parallel Computing*, second edition, Prentice Hall, Upper Saddle River, NJ.

An entertaining book that advocates clusters and is critical of NUMA multiprocessors.

Regnier, G., S. Makineni, I. Illikkal, R. Iyer, D. Minturn, R. Huggahalli, and A. Foong [2004]. TCP onloading for data center servers. *Computer*, 37(11), 48–58.

A paper describing benefits of doing TCP/IP inside servers vs. external hardware.

Seitz, C. [1985]. "The Cosmic Cube", *Comm. ACM* 28 1 (January), 22–31.

A tutorial article on a parallel processor connected via a hypertree. The Cosmic Cube is the ancestor of the Intel supercomputers.

Slotnick, D. L. [1982]. “The conception and development of parallel processors—a personal memoir”, *Annals of the History of Computing* 4: 1 (January), 20–30.

Recollections of the beginnings of parallel processing by the architect of the Illiac I V.

Williams, S., J. Carter, L. Oliker, J. Shalf, and K. Yelick [2008]. “Lattice Boltzmann simulation optimization on leading multicore platforms,” *International Parallel & Distributed Processing Symposium (IPDPS)*.

Paper containing the results of the four multicores for LBMHD.

Williams, S., L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel [2007]. “Optimization of sparse matrix-vector multiplication on emerging multicore platforms,” *Supercomputing (SC)*.

Paper containing the results of the four multicores for SPm V.

Williams, S. [2008]. *Autotuning Performance of Multicore Computers*, Ph.D. Dissertation, U.C. Berkeley.

Dissertation containing the roofline model.

Woo, S. C., M. Ohara, E. Torrie, J. P. Singh, and A. Gupta. “The SPLASH-2 programs: characterization and methodological considerations,” *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA '95)*, May, 24–36.

Paper describing the second version of the Stanford parallel benchmarks.

6.1.2 [5] <\$6.2> Next, consider which of the activities could be carried out concurrently (e.g., eating breakfast and listening to the news). For each of your activities, describe which other activity could be paired with this activity.

6.1.3 [5] <\$6.2> For Exercise 6.1.2, what could we change about current systems (e.g., showers, clothes, TVs, cars) so that we could perform more tasks in parallel?

6.1.4 [5] <\$6.2> Estimate how much shorter time it would take to carry out these activities if you tried to carry out as many tasks in parallel as possible.

6.2 You are trying to bake three blueberry pound cakes. Cake ingredients are as follows:

1 cup butter, softened
1 cup sugar
4 large eggs
1 teaspoon vanilla extract
1/2 teaspoon salt
1/4 teaspoon nutmeg
1 1/2 cups flour
1 cup blueberries

The recipe for a single cake is as follows:

Step 1: Preheat oven to 325°F (160°C). Grease and flour your cake pan.

Step 2: In large bowl, beat together with a mixer butter and sugar at medium speed until light and fluffy. Add eggs, vanilla, salt and nutmeg. Beat until thoroughly blended. Reduce mixer speed to low and add flour, 1/2 cup at a time, beating just until blended.

Step 3: Gently fold in blueberries. Spread evenly in prepared baking pan. Bake for 60 minutes.

6.2.1 [5] <\$6.2> Your job is to cook three cakes as efficiently as possible. Assuming that you only have one oven large enough to hold one cake, one large bowl, one cake pan, and one mixer, come up with a schedule to make three cakes as quickly as possible. Identify the bottlenecks in completing this task.

6.2.2 [5] <\$6.2> Assume now that you have three bowls, three cake pans and three mixers. How much faster is the process now that you have additional resources?

6.2.3 [5] <\$6.2> Assume now that you have two friends that will help you cook, and that you have a large oven that can accommodate all three cakes. How will this change the schedule you arrived at in Exercise 6.2.1 above?

6.2.4 [5] <\$6.2> Compare the cake-making task to computing three iterations of a loop on a parallel computer. Identify data-level parallelism and task-level parallelism in the cake-making loop.

6.3 Many computer applications involve searching through a set of data and sorting the data. A number of efficient searching and sorting algorithms have been devised in order to reduce the runtime of these tedious tasks. In this problem we will consider how best to parallelize these tasks.

6.3.1 [10] <§6.2> Consider the following binary search algorithm (a classic divide and conquer algorithm) that searches for a value X in a sorted N -element array A and returns the index of matched entry:

```

BinarySearch(A[0..N-1], X) {
    low = 0
    high = N - 1
    while (low <= high) {
        mid = (low + high) / 2
        if (A[mid] > X)
            high = mid - 1
        else if (A[mid] < X)
            low = mid + 1
        else
            return mid // found
    }
    return -1 // not found
}

```

Assume that you have Y cores on a multi-core processor to run `BinarySearch`. Assuming that Y is much smaller than N , express the speed-up factor you might expect to obtain for values of Y and N . Plot these on a graph.

6.3.2 [5] <§6.2> Next, assume that Y is equal to N . How would this affect your conclusions in your previous answer? If you were tasked with obtaining the best speed-up factor possible (i.e., strong scaling), explain how you might change this code to obtain it.

6.4 Consider the following piece of C code:

```

for (j=2; j<=1000; j++)
    D[j] = D[j-1] + D[j-2];

```

The ARMv8 code corresponding to the above fragment is:

```

MOV     X10, #8000
ADD     X2, X0, X10
ADDI    X1, X0, #16
LOOP:   LDUR    D0, [X1, #-16]
        LDUR    D2, [X1, #-8]
        FADD    D4, D0, D2

```

```

STUR    D4, [X1, #0]
ADDI    X1, X1, #8
CMP     X1, X2
B.LE    LOOP

```

The latency of an instruction is the number of cycles that must come between that instruction and an instruction using the result. Assume floating point instructions have the following associated latencies (in cycles):

FADD	LDUR	STUR
4	6	1

6.4.1 [10] <§6.2> How many cycles does it take to execute this code?

6.4.2 [10] <§6.2> Re-order the code to reduce stalls. Now, how many cycles does it take to execute this code? (Hint: You can remove additional stalls by changing the offset on the STUR instruction.)

6.4.3 [10] <§6.2> When an instruction in a later iteration of a loop depends upon a data value produced in an earlier iteration of the same loop, we say that there is a *loop-carried dependence* between iterations of the loop. Identify the loop-carried dependences in the above code. Identify the dependent program variable and assembly-level registers. You can ignore the loop induction variable *j*.

6.4.4 [15] <§6.2> Rewrite the code by using registers to carry the data between iterations of the loop (as opposed to storing and re-loading the data from main memory). Show where this code stalls and calculate the number of cycles required to execute.

6.4.5 [10] <§6.2> Loop unrolling was described in [Chapter 4](#). Unroll and optimize the loop above so that each unrolled loop handles three iterations of the original loop. Show where this code stalls and calculate the number of cycles required to execute.

6.4.6 [10] <§6.2> The unrolling from Exercise 6.4.5. works nicely because we happen to want a multiple of three iterations. What happens if the number of iterations is not known at compile time? How can we efficiently handle a number of iterations that isn't a multiple of the number of iterations per unrolled loop?

6.4.7 [15] <§6.2> Consider running this code on a two-node distributed memory message passing system. Assume that we are going to use message passing as described in Section 6.7, where we introduce a new operation `send(x, y)` that sends to node *x* the value *y*, and an operation `receive()` that waits for the value being sent to it. Assume that `send` operations take one cycle to issue (i.e., later instructions on the same node can proceed on the next cycle), but take several cycles to be received on the receiving node. Receive instructions stall execution on the node where they are executed until they receive a message. Can you use such a system to speed up the code for this exercise? If so, what is the maximum latency for receiving information that can be tolerated? If not, why not?

6.5 Consider the following recursive mergesort algorithm (another classic divide and conquer algorithm). Mergesort was first described by John Von Neumann in 1945. The basic idea is to divide an unsorted list x of m elements into two sublists of about half the size of the original list. Repeat this operation on each sublist, and continue until we have lists of size 1 in length. Then starting with sublists of length 1, “merge” the two sublists into a single sorted list.

```
Mergesort(m)
  var list left, right, result
  if length(m) ≤ 1
    return m
  else
    var middle = length(m) / 2
    for each x in m up to middle
      add x to left
    for each x in m after middle
      add x to right
    left = Mergesort(left)
    right = Mergesort(right)
    result = Merge(left, right)
    return result
```

The merge step is carried out by the following code:

```
Merge(left, right)
  var list result
  while length(left) > 0 and length(right) > 0
    if first(left) ≤ first(right)
      append first(left) to result
      left = rest(left)
    else
      append first(right) to result
      right = rest(right)
  if length(left) > 0
    append rest(left) to result
  if length(right) > 0
    append rest(right) to result
  return result
```

6.5.1 [10] <\$6.2> Assume that you have Y cores on a multicore processor to run Mergesort. Assuming that Y is much smaller than length (m), express the speed-up factor you might expect to obtain for values of Y and length (m). Plot these on a graph.

6.5.2 [10] <\$6.2> Next, assume that Y is equal to length (m). How would this affect your conclusions in your previous answer? If you were tasked with obtaining the best speed-up factor possible (i.e., strong scaling), explain how you might change this code to obtain it.

6.6 Matrix multiplication plays an important role in a number of applications. Two matrices can only be multiplied if the number of columns of the first matrix is equal to the number of rows in the second.

Let's assume we have an $m \times n$ matrix A and we want to multiply it by an $n \times p$ matrix B . We can express their product as an $m \times p$ matrix denoted by AB (or $A \cdot B$). If we assign $C = AB$, and c_{ij} denotes the entry in C at position (i, j) , then for each element i and j with $1 \leq i \leq m$ and $1 \leq j \leq p$ $c_{ij} = \sum_{k=1}^n a_{i,k} \times b_{k,j}$. Now we want to see if we can parallelize the computation of C . Assume that matrices are laid out in memory sequentially as follows: $a_{1,1}, a_{2,1}, a_{3,1}, a_{4,1}, \dots$, etc.

6.6.1 [10] <\$6.5> Assume that we are going to compute C on both a single-core shared-memory machine and a four-core shared-memory machine. Compute the speed-up we would expect to obtain on the four-core machine, ignoring any memory issues.

6.6.2 [10] <\$6.5> Repeat Exercise 6.6.1, assuming that updates to C incur a cache miss due to false sharing when consecutive elements are in a row (i.e., index i) are updated.

6.6.3 [10] <\$6.5> How would you fix the false sharing issue that can occur?

6.7 Consider the following portions of two different programs running at the same time on four processors in a *symmetric multicore processor* (SMP). Assume that before this code is run, both x and y are 0.

Core 1: $x = 2$;

Core 2: $y = 2$;

Core 3: $w = x + y + 1$;

Core 4: $z = x + y$;

6.7.1 [10] <\$6.5> What are all the possible resulting values of w, x, y , and z ? For each possible outcome, explain how we might arrive at those values. You will need to examine all possible interleavings of instructions.

6.7.2 [5] <\$6.5> How could you make the execution more deterministic so that only one set of values is possible?

6.8 The dining philosopher's problem is a classic problem of synchronization and concurrency. The general problem is stated as philosophers sitting at a round table doing one of two things: eating or thinking. When they are eating, they are not thinking, and when they are thinking, they are not eating. There is a bowl of pasta in the center. A fork is placed in between each philosopher. The result is that each philosopher has one fork to her left and one fork to her right. Given the nature of eating pasta, the philosopher needs two forks to eat, and can only use the forks on her immediate left and right. The philosophers do not speak to one another.

6.8.1 [10] <\$6.7> Describe the scenario where none of philosophers ever eats (i.e., starvation). What is the sequence of events that happen that lead up to this problem?

6.8.2 [10] <\$6.7> Describe how we can solve this problem by introducing the concept of a priority. Can we guarantee that we will treat all the philosophers fairly? Explain.

Now assume we hire a waiter who is in charge of assigning forks to philosophers. Nobody can pick up a fork until the waiter says they can. The waiter has global knowledge of all forks. Further, if we impose the policy that philosophers will always request to pick up their left fork before requesting to pick up their right fork, then we can guarantee to avoid deadlock.

6.8.3 [10] <\$6.7> We can implement requests to the waiter as either a queue of requests or as a periodic retry of a request. With a queue, requests are handled in the order they are received. The problem with using the queue is that we may not always be able to service the philosopher whose request is at the head of the queue (due to the unavailability of resources). Describe a scenario with five philosophers where a queue is provided, but service is not granted even though there are forks available for another philosopher (whose request is deeper in the queue) to eat.

6.8.4 [10] <\$6.7> If we implement requests to the waiter by periodically repeating our request until the resources become available, will this solve the problem described in Exercise 6.8.3? Explain.

6.9 Consider the following three CPU organizations:

CPU SS: A two-core superscalar microprocessor that provides out-of-order issue capabilities on two *function units* (FUs). Only a single thread can run on each core at a time.

CPU MT: A fine-grained multithreaded processor that allows instructions from two threads to be run concurrently (i.e., there are two functional units), though only instructions from a single thread can be issued on any cycle.

CPU SMT: An SMT processor that allows instructions from two threads to be run concurrently (i.e., there are two functional units), and instructions from either or both threads can be issued to run on any cycle.

Assume we have two threads X and Y to run on these CPUs that include the following operations:

Thread X	Thread Y
A1 – takes three cycles to execute	B1 – take two cycles to execute
A2 – no dependences	B2 – conflicts for a functional unit with B1
A3 – conflicts for a functional unit with A1	B3 – depends on the result of B2
A4 – depends on the result of A3	B4 – no dependences and takes two cycles to execute

Assume all instructions take a single cycle to execute unless noted otherwise or they encounter a hazard.

6.9.1 [10] <\$6.4> Assume that you have one SS CPU. How many cycles will it take to execute these two threads? How many issue slots are wasted due to hazards?

6.9.2 [10] <\$6.4> Now assume you have two SS CPUs. How many cycles will it take to execute these two threads? How many issue slots are wasted due to hazards?

6.9.3 [10] <\$6.4> Assume that you have one MT CPU. How many cycles will it take to execute these two threads? How many issue slots are wasted due to hazards?

6.9.4 [10] <\$6.4> Assume you have one SMT CPU. How many cycles will it take to execute the two threads? How many issue slots are wasted due to hazards?

6.10 Virtualization software is being aggressively deployed to reduce the costs of managing today's high-performance servers. Companies like VMWare, Microsoft and IBM have all developed a range of virtualization products. The general concept, described in [Chapter 5](#), is that a hypervisor layer can be introduced between the hardware and the operating system to allow multiple operating systems to share the same physical hardware. The hypervisor layer is then responsible for allocating CPU and memory resources, as well as handling services typically handled by the operating system (e.g., I/O).

Virtualization provides an abstract view of the underlying hardware to the hosted operating system and application software. This will require us to rethink how multi-core and multiprocessor systems will be designed in the future to support the sharing of CPUs and memories by a number of operating systems concurrently.

6.10.1 [30] <\$6.4> Select two hypervisors on the market today, and compare and contrast how they virtualize and manage the underlying hardware (CPUs and memory).

6.10.2 [15] <\$6.4> Discuss what changes may be necessary in future multi-core CPU platforms in order to better match the resource demands placed on these systems. For instance, can multithreading play an effective role in alleviating the competition for computing resources?

6.11 We would like to execute the loop below as efficiently as possible. We have two different machines, a MIMD machine and a SIMD machine.

```
for (i=0; i<2000; i++)
    for (j=0; j<3000; j++)
        X_array[i][j] = Y_array[j][i] + 200;
```

6.11.1 [10] <\$6.3> For a four CPU MIMD machine, show the sequence of LEGv8 instructions that you would execute on each CPU. What is the speed-up for this MIMD machine?

6.11.2 [20] <§6.3> For an eight-wide SIMD machine (i.e., eight parallel SIMD functional units), write an assembly program in using your own SIMD extensions to LEGv8 to execute the loop. Compare the number of instructions executed on the SIMD machine to the MIMD machine.

6.12 A systolic array is an example of an MISD machine. A systolic array is a pipeline network or “wavefront” of data processing elements. Each of these elements does not need a program counter since execution is triggered by the arrival of data. Clocked systolic arrays compute in “lock-step” with each processor undertaking alternate compute and communication phases.

6.12.1 [10] <§6.3> Consider proposed implementations of a systolic array (you can find these on the Internet or in technical publications). Then attempt to program the loop provided in Exercise 6.11 using this MISD model. Discuss any difficulties you encounter.

6.12.2 [10] <§6.3> Discuss the similarities and differences between an MISD and SIMD machines. Answer this question in terms of data-level parallelism.

6.13 Assume we want to execute the DAXP loop shown on page 526 in ARMv8t assembly on the NVIDIA 8800 GTX GPU described in this chapter. In this problem, we will assume that all math operations are performed on single-precision floating-point numbers (we will rename the loop SAXPY). Assume that instructions take the following number of cycles to execute.

Loads	Stores	Add.S	Mult.S
5	2	3	4

6.13.1 [20] <§6.6> Describe how you will constructs warps for the SAXPY loop to exploit the eight cores provided in a single multiprocessor.

6.14 Download the CUDA Toolkit and SDK from <https://developer.nvidia.com/cuda-toolkit>. Make sure to use the “emurelease” (Emulation Mode) version of the code. (You will not need actual NVIDIA hardware for this assignment.) Build the example programs provided in the SDK, and confirm that they run on the emulator.

6.14.1 [90] <§6.6> Using the “template” SDK sample as a starting point, write a CUDA program to perform the following vector operations:

- 1) $a - b$ (vector-vector subtraction)
- 2) $a \cdot b$ (vector dot product)

The dot product of two vectors $a = [a_1, a_2, \dots, a_n]$ and $b = [b_1, b_2, \dots, b_n]$ is defined as:

$$a \cdot b = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

Submit code for each program that demonstrates each operation and verifies the correctness of the results.

6.14.2 [90] <\$6.6> If you have GPU hardware available, complete a performance analysis on your program, examining the computation time for the GPU and a CPU version of your program for a range of vector sizes. Explain any results you see.

6.15 AMD has recently announced integrating a graphics processing unit with their x86 cores into a single package (though with different clocks for each of the cores). This is an example of a heterogeneous multiprocessor system. One of the key design points is to allow for fast data communication between the CPU and the GPU. Before AMD's Fusion architecture, communications were needed between discrete CPU and GPU chips. Presently, the plan is to use multiple (at least 16) PCI express channels to facilitate intercommunication.

6.15.1 [25] <\$6.6> Compare the bandwidth and latency associated with these two interconnect technologies.

6.16 Refer to [Figure 6.14b](#), which shows an n-cube interconnect topology of order 3 that interconnects eight nodes. One attractive feature of an n-cube interconnection network topology is its ability to sustain broken links and still provide connectivity.

6.16.1 [10] <\$6.8> Develop an equation that computes how many links in the n-cube (where n is the order of the cube) can fail and we can still guarantee an unbroken link will exist to connect any node in the n-cube.

6.16.2 [10] <\$6.8> Compare the resiliency to failure of n-cube to a fully connected interconnection network. Plot a comparison of reliability as a function of the added number of links for the two topologies.

6.17 Benchmarking is a field of study that involves identifying representative workloads to run on specific computing platforms in order to be able to objectively compare performance of one system to another. In this exercise we will compare two classes of benchmarks: the Whetstone CPU benchmark and the PARSEC Benchmark suite. Select one program from PARSEC. All programs should be freely available on the Internet. Consider running multiple copies of Whetstone versus running the PARSEC Benchmark on any of the systems described in Section 6.11.

6.17.1 [60] <\$6.10> What is inherently different between these two classes of workload when run on these multi-core systems?

6.17.2 [60] <\$6.10> In terms of the Roofline Model, how dependent will the results you obtain when running these benchmarks be on the amount of sharing and synchronization present in the workload used?

6.18 When performing computations on sparse matrices, latency in the memory hierarchy becomes much more of a factor. Sparse matrices lack the spatial locality in the datastream typically found in matrix operations. As a result, new matrix representations have been proposed.

One of the earliest sparse matrix representations is the Yale Sparse Matrix Format. It stores an initial sparse $m \times n$ matrix, M in row form using three one-dimensional arrays. Let R be the number of nonzero entries in M . We construct an array A of length R that contains all nonzero entries of M (in left-to-right top-to-bottom order). We also construct a second array IA of length $m+1$ (i.e., one entry per row, plus one). $IA(i)$ contains the index in A of the first nonzero element of row i . Row i of the original matrix extends from $A(IA(i))$ to $A(IA(i+1)-1)$. The third array, JA , contains the column index of each element of A , so it also is of length R .

6.18.1 [15] <\$6.10> Consider the sparse matrix X below and write C code that would store this code in Yale Sparse Matrix Format.

```
Row 1 [1, 2, 0, 0, 0, 0]
Row 2 [0, 0, 1, 1, 0, 0]
Row 3 [0, 0, 0, 0, 9, 0]
Row 4 [2, 0, 0, 0, 0, 2]
Row 5 [0, 0, 3, 3, 0, 7]
Row 6 [1, 3, 0, 0, 0, 1]
```

6.18.2 [10] <\$6.10> In terms of storage space, assuming that each element in matrix X is single-precision floating point, compute the amount of storage used to store the matrix above in Yale Sparse Matrix Format.

6.18.3 [15] <\$6.10> Perform matrix multiplication of matrix X by matrix Y shown below.

```
[2, 4, 1, 99, 7, 2]
```

Put this computation in a loop, and time its execution. Make sure to increase the number of times this loop is executed to get good resolution in your timing measurement. Compare the runtime of using a naïve representation of the matrix, and the Yale Sparse Matrix Format.

6.18.4 [15] <\$6.10> Can you find a more efficient sparse matrix representation (in terms of space and computational overhead)?

6.19 In future systems, we expect to see heterogeneous computing platforms constructed out of heterogeneous CPUs. We have begun to see some appear in the embedded processing market in systems that contain both floating-point DSPs and microcontroller CPUs in a multichip module package.

Assume that you have three classes of CPU:

CPU A—A moderate-speed multi-core CPU (with a floating-point unit) that can execute multiple instructions per cycle.

CPU B—A fast single-core integer CPU (i.e., no floating-point unit) that can execute a single instruction per cycle.

CPU C—A slow vector CPU (with floating-point capability) that can execute multiple copies of the same instruction per cycle.

Assume that our processors run at the following frequencies:

CPU A	CPU B	CPU C
1 GHz	3 GHz	250 MHz

CPU A can execute two instructions per cycle, CPU B can execute one instruction per cycle, and CPU C can execute eight instructions (through the same instruction) per cycle. Assume all operations can complete execution in a single cycle of latency without any hazards.

All three CPUs have the ability to perform integer arithmetic, though CPU B cannot perform floating point arithmetic. CPU A and B have an instruction set similar to a LEGv8 processor. CPU C can only perform floating point add and subtract operations, as well as memory loads and stores. Assume all CPUs have access to shared memory and that synchronization has zero cost.

The task at hand is to compare two matrices X and Y that each contain 1024×1024 floating-point elements. The output should be a count of the number of indices where the value in X was larger or equal to the value in Y.

6.19.1 [10] <\$6.11> Describe how you would partition the problem on the three different CPUs to obtain the best performance.

6.19.2 [10] <\$6.11> What kind of instruction would you add to the vector CPU C to obtain better performance?

6.20 This question looks at the amount of queuing that is occurring in the system given a maximum transaction processing rate, and the latency observed on average for a transaction. The latency includes both the service time (which is computed by the maximum rate) and the queue time.

Assume a quad-core computer system can process database queries at a steady state maximum rate of rate requests per second. Also assume that each transaction takes, on average, lat ms to process. For each of the pairs in the table, answer the following questions:

Average Transaction Latency	Maximum transaction processing rate
1 ms	5000/sec
2 ms	5000/sec
1 ms	10,000/sec
2 ms	10,000/sec

For each of the pairs in the table, answer the following questions:

6.20.1 [10] <\$6.11> On average, how many requests are being processed at any given instant?

6.20.2 [10] <\$6.11> If we move to an eight-core system, ideally, what will happen to the system throughput (i.e., how many queries/second will the computer process)?

6.20.3 [10] <\$6.11> Discuss why we rarely obtain this kind of speed-up by simply increasing the number of cores.

**Answers to
Check Yourself**

§6.1, page 518: False. Task-level parallelism can help sequential applications and sequential applications can be made to run on parallel hardware, although it is more challenging.

§6.2, page 523: False. *Weak* scaling can compensate for a serial portion of the program that would otherwise limit scalability, but not so for strong scaling.

§6.3, page 528: True, but they are missing useful vector features like gather-scatter and vector length registers that improve the efficiency of vector architectures. (As an elaboration in this section mentions, the AVX2 SIMD extensions offers indexed loads via a gather operation but *not* scatter for indexed stores. The Haswell generation x86 microprocessor is the first to support AVX2.)

§6.4, page 533: 1. True. 2. True.

§6.5, page 537: False. Since the shared address is a *physical* address, multiple tasks each in their own *virtual* address spaces can run well on a shared memory multiprocessor.

§6.6, page 545: False. Graphics DRAM chips are prized for their higher bandwidth.

§6.7, page 550: 1. False. Sending and receiving a message is an implicit synchronization, as well as a way to share data. 2. True.

§6.8, page 552: True.

§6.9, page 6.9-10: 1. polling with loads and stores. 2. interrupts with DMA.

§6.10, page 564: True. We likely need innovation at all levels of the hardware and software stack for parallel computing to succeed.



A P P E N D I X

*I always loved that
word, Boolean.*

Claude Shannon

IEEE Spectrum, April 1992
(Shannon's master's thesis showed
that the algebra invented by George
Boole in the 1800s could represent the
workings of electrical switches.)

The Basics of Logic Design

- A.1 Introduction** A-3
- A.2 Gates, Truth Tables, and Logic
Equations** A-4
- A.3 Combinational Logic** A-9
- A.4 Using a Hardware Description
Language** A-20
- A.5 Constructing a Basic Arithmetic Logic
Unit** A-26
- A.6 Faster Addition: Carry Lookahead** A-37
- A.7 Clocks** A-47

A.8	Memory Elements: Flip-Flops, Latches, and Registers	A-49
A.9	Memory Elements: SRAMs and DRAMs	A-57
A.10	Finite-State Machines	A-66
A.11	Timing Methodologies	A-71
A.12	Field Programmable Devices	A-77
A.13	Concluding Remarks	A-78
A.14	Exercises	A-79

A.1

Introduction

This appendix provides a brief discussion of the basics of logic design. It does not replace a course in logic design, nor will it enable you to design significant working logic systems. If you have little or no exposure to logic design, however, this appendix will provide sufficient background to understand all the material in this book. In addition, if you are looking to understand some of the motivation behind how computers are implemented, this material will serve as a useful introduction. If your curiosity is aroused but not sated by this appendix, the references at the end provide several additional sources of information.

Section A.2 introduces the basic building blocks of logic, namely, *gates*. Section A.3 uses these building blocks to construct simple *combinational* logic systems, which contain no memory. If you have had some exposure to logic or digital systems, you will probably be familiar with the material in these first two sections. Section A.5 shows how to use the concepts of Sections A.2 and A.3 to design an ALU for the LEGv8 processor. Section A.6 shows how to make a fast adder, and

may be safely skipped if you are not interested in this topic. Section A.7 is a short introduction to the topic of clocking, which is necessary to discuss how memory elements work. Section A.8 introduces memory elements, and Section A.9 extends it to focus on random access memories; it describes both the characteristics that are important to understanding how they are used, as discussed in [Chapter 4](#), and the background that motivates many of the aspects of memory hierarchy design discussed in [Chapter 5](#). Section A.10 describes the design and use of finite-state machines, which are sequential logic blocks. If you intend to read [Appendix C](#), you should thoroughly understand the material in Sections A.2 through A.10. If you intend to read only the material on control in [Chapter 4](#), you can skim the appendices; however, you should have some familiarity with all the material except Section A.11. Section A.11 is intended for those who want a deeper understanding of clocking methodologies and timing. It explains the basics of how edge-triggered clocking works, introduces another clocking scheme, and briefly describes the problem of synchronizing asynchronous inputs.

Throughout this appendix, where it is appropriate, we also include segments to demonstrate how logic can be represented in Verilog, which we introduce in Section A.4. A more extensive and complete Verilog tutorial is available online on the Companion Web site for this book.

A.2

Gates, Truth Tables, and Logic Equations

The electronics inside a modern computer are *digital*. Digital electronics operate with only two voltage levels of interest: a high voltage and a low voltage. All other voltage values are temporary and occur while transitioning between the values. (As we discuss later in this section, a possible pitfall in digital design is sampling a signal when it not clearly either high or low.) The fact that computers are digital is also a key reason they use binary numbers, since a binary system matches the underlying abstraction inherent in the electronics. In various logic families, the values and relationships between the two voltage values differ. Thus, rather than refer to the voltage levels, we talk about signals that are (logically) true, or 1, or are **asserted**; or signals that are (logically) false, or 0, or are **deasserted**. The values 0 and 1 are called *complements* or *inverses* of one another.

Logic blocks are categorized as one of two types, depending on whether they contain memory. Blocks without memory are called *combinational*; the output of a combinational block depends only on the current input. In blocks with memory, the outputs can depend on both the inputs and the value stored in memory, which is called the *state* of the logic block. In this section and the next, we will focus

asserted signal A signal that is (logically) true, or 1.

deasserted signal
A signal that is (logically) false, or 0.

only on **combinational logic**. After introducing different memory elements in Section A.8, we will describe how **sequential logic**, which is logic including state, is designed.

Truth Tables

Because a combinational logic block contains no memory, it can be completely specified by defining the values of the outputs for each possible set of input values. Such a description is normally given as a *truth table*. For a logic block with n inputs, there are 2^n entries in the truth table, since there are that many possible combinations of input values. Each entry specifies the value of all the outputs for that particular input combination.

combinational logic

A logic system whose blocks do not contain memory and hence compute the same output given the same input.

sequential logic

A group of logic elements that contain memory and hence whose value depends on the inputs as well as the current contents of the memory.

Truth Tables

Consider a logic function with three inputs, A , B , and C , and three outputs, D , E , and F . The function is defined as follows: D is true if at least one input is true, E is true if exactly two inputs are true, and F is true only if all three inputs are true. Show the truth table for this function.

The truth table will contain $2^3 = 8$ entries. Here it is:

Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

Truth tables can completely describe any combinational logic function; however, they grow in size quickly and may not be easy to understand. Sometimes we want to construct a logic function that will be 0 for many input combinations, and we use a shorthand of specifying only the truth table entries for the nonzero outputs. This approach is used in [Chapter 4](#) and [Appendix C](#).

EXAMPLE

ANSWER

Boolean Algebra

Another approach is to express the logic function with logic equations. This is done with the use of *Boolean algebra* (named after Boole, a 19th-century mathematician). In Boolean algebra, all the variables have the values 0 or 1 and, in typical formulations, there are three operators:

- The OR operator is written as $+$, as in $A + B$. The result of an OR operator is 1 if either of the variables is 1. The OR operation is also called a *logical sum*, since its result is 1 if either operand is 1.
- The AND operator is written as $\cdot$, as in $A \cdot B$. The result of an AND operator is 1 only if both inputs are 1. The AND operator is also called *logical product*, since its result is 1 only if both operands are 1.
- The unary operator NOT is written as $\bar{A}$. The result of a NOT operator is 1 only if the input is 0. Applying the operator NOT to a logical value results in an inversion or negation of the value (i.e., if the input is 0 the output is 1, and vice versa).

There are several laws of Boolean algebra that are helpful in manipulating logic equations.

- Identity law: $A + 0 = A$ and $A \cdot 1 = A$
- Zero and One laws: $A + 1 = 1$ and $A \cdot 0 = 0$
- Inverse laws: $A + \bar{A} = 1$ and $A \cdot \bar{A} = 0$
- Commutative laws: $A + B = B + A$ and $A \cdot B = B \cdot A$
- Associative laws: $A + (B + C) = (A + B) + C$ and $A \cdot (B \cdot C) = (A \cdot B) \cdot C$
- Distributive laws: $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$ and $A + (B \cdot C) = (A + B) \cdot (A + C)$

In addition, there are two other useful theorems, called DeMorgan's laws, that are discussed in more depth in the exercises.

Any set of logic functions can be written as a series of equations with an output on the left-hand side of each equation and a formula consisting of variables and the three operators above on the right-hand side.

Logic Equations

Show the logic equations for the logic functions, D , E , and F , described in the previous example.

EXAMPLE

Here's the equation for D :

$$D = A + B + C$$

F is equally simple:

$$F = A \cdot B \cdot C$$

ANSWER

E is a little tricky. Think of it in two parts: what must be true for E to be true (two of the three inputs must be true), and what cannot be true (all three cannot be true). Thus we can write E as

$$E = ((A \cdot B) + (A \cdot C) + (B \cdot C)) \cdot \overline{(A \cdot B \cdot C)}$$

We can also derive E by realizing that E is true only if exactly two of the inputs are true. Then we can write E as an OR of the three possible terms that have two true inputs and one false input:

$$E = (A \cdot B \cdot \bar{C}) + (A \cdot C \cdot \bar{B}) + (B \cdot C \cdot \bar{A})$$

Proving that these two expressions are equivalent is explored in the exercises.

In Verilog, we describe combinational logic whenever possible using the assign statement, which is described beginning on page A-23. We can write a definition for E using the Verilog exclusive-OR operator as `assign E = (A ^ B ^ C) * (A + B + C) * (A * B * C)`, which is yet another way to describe this function. D and F have even simpler representations, which are just like the corresponding C code: $D = A \mid B \mid C$ and $F = A \& B \& C$.

gate A device that implements basic logic functions, such as AND or OR.

NOR gate An inverted OR gate.

NAND gate An inverted AND gate.

Check Yourself

Gates

Logic blocks are built from **gates** that implement basic logic functions. For example, an AND gate implements the AND function, and an OR gate implements the OR function. Since both AND and OR are commutative and associative, an AND or an OR gate can have multiple inputs, with the output equal to the AND or OR of all the inputs. The logical function NOT is implemented with an inverter that always has a single input. The standard representation of these three logic building blocks is shown in **Figure A.2.1**.

Rather than draw inverters explicitly, a common practice is to add “bubbles” to the inputs or outputs of a gate to cause the logic value on that input line or output line to be inverted. For example, **Figure A.2.2** shows the logic diagram for the function $\overline{A + B}$, using explicit inverters on the left and bubbled inputs and outputs on the right.

Any logical function can be constructed using AND gates, OR gates, and inversion; several of the exercises give you the opportunity to try implementing some common logic functions with gates. In the next section, we’ll see how an implementation of any logic function can be constructed using this knowledge.

In fact, all logic functions can be constructed with only a single gate type, if that gate is inverting. The two common inverting gates are called **NOR** and **NAND** and correspond to inverted OR and AND gates, respectively. NOR and NAND gates are called *universal*, since any logic function can be built using this one gate type. The exercises explore this concept further.

Are the following two logical expressions equivalent? If not, find a setting of the variables to show they are not:

- $(A \cdot B \cdot \overline{C}) + (A \cdot C \cdot \overline{B}) + (B \cdot C \cdot \overline{A})$
- $B \cdot (A \cdot \overline{C} + C \cdot \overline{A})$



FIGURE A.2.1 Standard drawing for an AND gate, OR gate, and an inverter, shown from left to right. The signals to the left of each symbol are the inputs, while the output appears on the right. The AND and OR gates both have two inputs. Inverters have a single input.

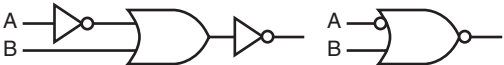


FIGURE A.2.2 Logic gate implementation of $\overline{A + B}$ using explicit inverters on the left and bubbled inputs and outputs on the right. This logic function can be simplified to $A \cdot \overline{B}$ or in Verilog, `A & ~ B`.

A.3

Combinational Logic

In this section, we look at a couple of larger logic building blocks that we use heavily, and we discuss the design of structured logic that can be automatically implemented from a logic equation or truth table by a translation program. Last, we discuss the notion of an array of logic blocks.

Decoders

One logic block that we will use in building larger components is a **decoder**. The most common type of decoder has an n -bit input and 2^n outputs, where only one output is asserted for each input combination. This decoder translates the n -bit input into a signal that corresponds to the binary value of the n -bit input. The outputs are thus usually numbered, say, Out0, Out1, ..., Out $2^n - 1$. If the value of the input is i , then Out i will be true and all other outputs will be false. [Figure A.3.1](#) shows a 3-bit decoder and the truth table. This decoder is called a *3-to-8 decoder* since there are three inputs and eight (2^3) outputs. There is also a logic element called an *encoder* that performs the inverse function of a decoder, taking 2^n inputs and producing an n -bit output.

decoder A logic block that has an n -bit input and 2^n outputs, where only one output is asserted for each input combination.

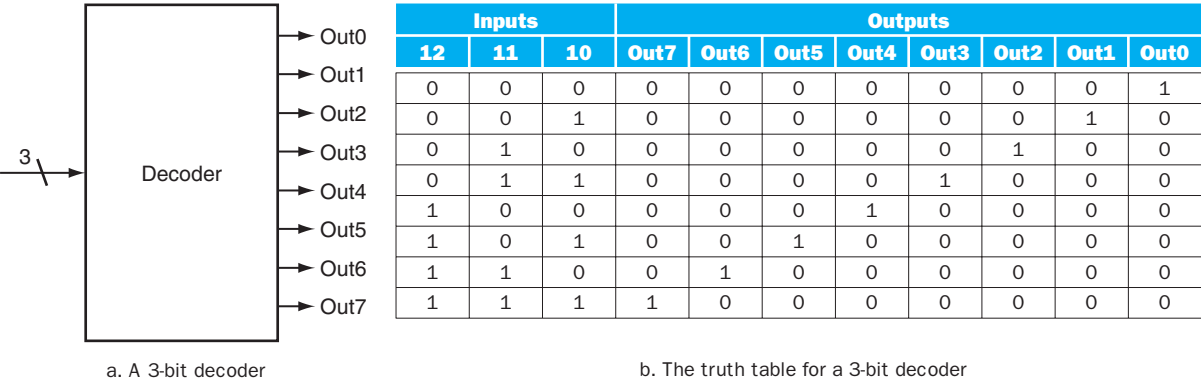


FIGURE A.3.1 A 3-bit decoder has three inputs, called 12, 11, and 10, and $2^3 = 8$ outputs, called Out0 to Out7. Only the output corresponding to the binary value of the input is true, as shown in the truth table. The label 3 on the input to the decoder says that the input signal is 3 bits wide.

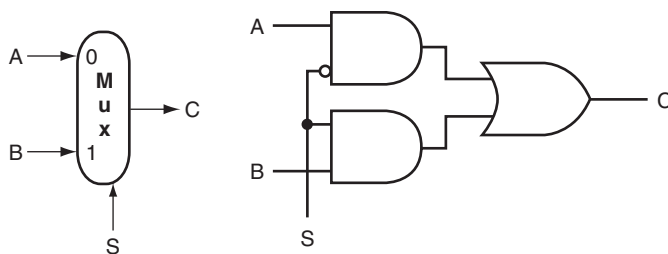


FIGURE A.3.2 A two-input multiplexor on the left and its implementation with gates on the right. The multiplexor has two data inputs (A and B), which are labeled 0 and 1, and one selector input (S), as well as an output C . Implementing multiplexors in Verilog requires a little more work, especially when they are wider than two inputs. We show how to do this beginning on page A-23.

Multiplexors

One basic logic function that we use quite often in [Chapter 4](#) is the *multiplexor*. A multiplexor might more properly be called a *selector*, since its output is one of the inputs that is selected by a control. Consider the two-input multiplexor. The left side of [Figure A.3.2](#) shows this multiplexor has three inputs: two data values and a **selector** (or **control**) **value**. The selector value determines which of the inputs becomes the output. We can represent the logic function computed by a two-input multiplexor, shown in gate form on the right side of [Figure A.3.2](#), as $C = (A \cdot S) + (B \cdot S)$.

Multiplexors can be created with an arbitrary number of data inputs. When there are only two inputs, the selector is a single signal that selects one of the inputs if it is true (1) and the other if it is false (0). If there are n data inputs, there will need to be $\lceil \log_2 n \rceil$ selector inputs. In this case, the multiplexor basically consists of three parts:

1. A decoder that generates n signals, each indicating a different input value
2. An array of n AND gates, each combining one of the inputs with a signal from the decoder
3. A single large OR gate that incorporates the outputs of the AND gates

To associate the inputs with selector values, we often label the data inputs numerically (i.e., 0, 1, 2, 3, ..., $n - 1$) and interpret the data selector inputs as a binary number. Sometimes, we make use of a multiplexor with undecoded selector signals.

Multiplexors are easily represented combinationally in Verilog by using *if* expressions. For larger multiplexors, *case* statements are more convenient, but care must be taken to synthesize combinational logic.

selector value Also called **control value**. The control signal that is used to select one of the input values of a multiplexor as the output of the multiplexor.

Two-Level Logic and PLAs

As pointed out in the previous section, any logic function can be implemented with only AND, OR, and NOT functions. In fact, a much stronger result is true. Any logic function can be written in a canonical form, where every input is either a true or complemented variable and there are only two levels of gates—one being AND and the other OR—with a possible inversion on the final output. Such a representation is called a *two-level representation*, and there are two forms, called **sum of products** and *product of sums*. A sum-of-products representation is a logical sum (OR) of products (terms using the AND operator); a product of sums is just the opposite. In our earlier example, we had two equations for the output E :

$$E = ((A \cdot B) + (A \cdot C) + (B \cdot C)) \cdot \overline{(A \cdot B \cdot C)}$$

and

$$E = (A \cdot B \cdot \bar{C}) + (A \cdot C \cdot \bar{B}) + (B \cdot C \cdot \bar{A})$$

This second equation is in a sum-of-products form: it has two levels of logic and the only inversions are on individual variables. The first equation has three levels of logic.

Elaboration: We can also write E as a product of sums:

$$E = \overline{(\bar{A} + \bar{B} + C)} \cdot \overline{(\bar{A} + \bar{C} + B)} \cdot \overline{(\bar{B} + C + A)}$$

To derive this form, you need to use *DeMorgan's theorems*, which are discussed in the exercises.

In this text, we use the sum-of-products form. It is easy to see that any logic function can be represented as a sum of products by constructing such a representation from the truth table for the function. Each truth table entry for which the function is true corresponds to a product term. The product term consists of a logical product of all the inputs or the complements of the inputs, depending on whether the entry in the truth table has a 0 or 1 corresponding to this variable. The logic function is the logical sum of the product terms where the function is true. This is more easily seen with an example.

sum of products A form of logical representation that employs a logical sum (OR) of products (terms joined using the AND operator).